existence of something that isn't there aren't straightforwardly false, but are rather defective in some other way — perhaps they fail to be meaningful at all, or perhaps they take on some truth value other than true or false. These matters are beyond our present scope. We will remain neutral on whether Russell's theory is an accurate treatment of English; it is relevant for us because it provides an interesting and useful case for translation of sentences into QL with identity.

The general statement of Russell's theory of definite descriptions is, if you have a sentence applying a predicate $\mathcal{P}$ to the referent of a definite description $\mathcal{D}$, translate it thus: there is something χ that is $\mathcal{D}$, there is nothing other than χ that is $\mathcal{D}$, and χ is $\mathcal{P}$.

12.7 Identity and trees

Because we have extended QL to include claims of identity, we also need to modify our tree system. Notice that if we do not do so, the system will not be complete. For example, the rules offered in Chapter 11 do not provide a way to demonstrate that $Fa, \neg Fb, a{=}b \models \bot$, even though those sentences are obviously inconsistent. Since each of those sentences is an atom or negated atom in QL, there is nothing to resolve, and there is no contradiction to close the tree. We need to add new rules to deal with identity. We'll make three modifications: one for new ways to develop branches, one for branch closure, and one for branch completion.

12.7.1 Developing branches with identity

To characterize our new branch development rule, we'll first define a notion of IDENTITY IN A BRANCH. The intuitive idea of identity in a branch is simple: names are identified in a branch if the branch contains identity claims that require them to refer to the same object. So, for example, if $a{=}b$ is in the branch, then a and b are identified. If $a{=}b$ and $b{=}c$ are both in the branch, then a, b, and c are all identified. Syntactically, we can define identity in a branch thus:

> Names $\mathcal{a}$ and $\mathcal{b}$ are IDENTIFIED ON A BRANCH if they are the same name, or if there is a chain of identity claims in the branch linking them. More precisely, $\mathcal{a}$ and $\mathcal{b}$ are identified on a branch if the branch contains identity claims that can be arranged in a sequence
>
> $$\mathcal{a}{=}\mathcal{c}_1, \quad \mathcal{c}_1{=}\mathcal{c}_2, \quad \ldots, \quad \mathcal{c}_n{=}\mathcal{b},$$
>
> where an identity claim may be read in either direction.

And here is an obvious rule for branch development with identity:

> If names $\mathcal{a}$ and $\mathcal{b}$ are identified in a branch, for any sentence Φ in the branch, then you may add $\Phi\boxed{\mathcal{a} \Rightarrow \mathcal{b}}$ to the branch.

Recall that '$\Phi\boxed{\mathcal{a} \Rightarrow \mathcal{b}}$' just means the sentence you get by replacing every '$\mathcal{a}$' in Φ with '$\mathcal{b}$'. Note also that the definition of identity in a branch is symmetric; if $\mathcal{a}$ is identified with $\mathcal{b}$ in a branch, then $\mathcal{b}$ is identified with $\mathcal{a}$. So this rule allows substitution in either direction.

With this rule, we can easily demonstrate that the set of sentences mentioned above closes:

$\{Fa, \neg Fb, a{=}b\} \vdash \bot$

1. Fa
2. $\neg Fb$
3. $a{=}b$
4. Fb — 1, 3 =

   $\times$ 2, 4

We will add this rule to our tree system. Notice that it is not a *resolution* rule — we do not put a check mark next to either sentence referenced. They may be used again.

12.7.2 Closing branches with identity

Previously we said that branches only close when they contain some sentence along with its negation. We add one more condition that suffices for branch closure. If any branch contains a sentence of the form $\mathcal{a}{\neq}\mathcal{a}$, then that branch closes. In no model can something be non-identical with itself.

Adding this to the rule given on page 215, we get:

> A branch closes if and only if either (i) it contains some sentence Φ along with its negation, $\neg\Phi$, or (ii) it contains, for some name $\mathcal{a}$, the sentence $\mathcal{a}{\neq}\mathcal{a}$.

We need this rule, for example, to prove with a tree that $\forall x\, a{=}x$ is inconsistent with $\exists x\, x{\neq}a$:

$\{\forall x\ a{=}x, \exists x\ x{\neq}a\} \vdash \bot$

1.	$\forall x\ a{=}x\ \backslash b$	
2.	$\exists x\ x{\neq}a\ \checkmark b$	
	$\vert$	
3.	$b{\neq}a$	2 $\exists$
	$\vert$	
4.	$a{=}b$	1 $\forall$
	$\vert$	
5.	$b{\neq}b$	3, 4 $=$
	$\times$ 5	

12.7.3 Identity and open branch completion

Since our tree system is supposed to be *complete*, we need to *ensure* that the tree will close; it's not enough that we have a rule that allows that it might. So, as in the case of the general rules discussed in Chapter 10, we will also add an additional requirement governing branch completion. What we need is to make sure that we've made enough identity substitutions to demonstrate everything that needs demonstrating. For example, this shouldn't count as a completed open branch:

1.	$\neg Fb$	
2.	Fa	
3.	Ga	
4.	$a{=}b$	
	$\vert$	
5.	Gb	3, 4 $=$
	$\uparrow$	

The development on line 5 is a good one, but we can't stop the tree there: in addition to substituting the a with a b in the Ga on line 3, we also need to make the same substitution in the Fa on line 2. That would give us Fb, and the tree will close. We need a branch completion rule that ensures that an open branch isn't complete until every necessary substitution has been made.

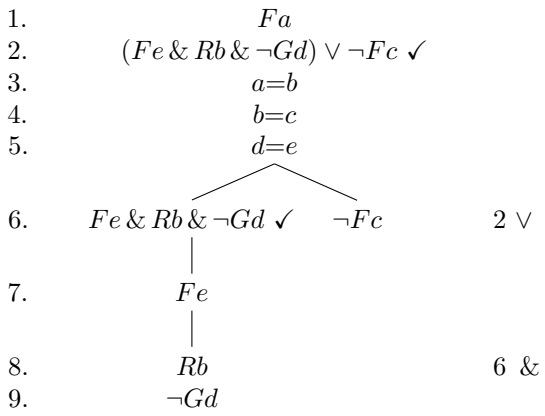
To articulate that rule, we'll use the notion of an IDENTITY GROUP. The relation of being identified on a branch, introduced in §12.7.1, divides the names in that branch into groups. Call each maximal set of names identified with one another an identity group. (A name that is not identified with any different name forms an identity group by itself.) For each identity group, choose one name in that group as its REPRESENTATIVE. The REPRESENTATIVE VERSION of an atomic or

negated atomic sentence is the sentence obtained by replacing every name in it with the representative of its identity group. We can revise our definition of branch completion, previously given on page 216, thus:

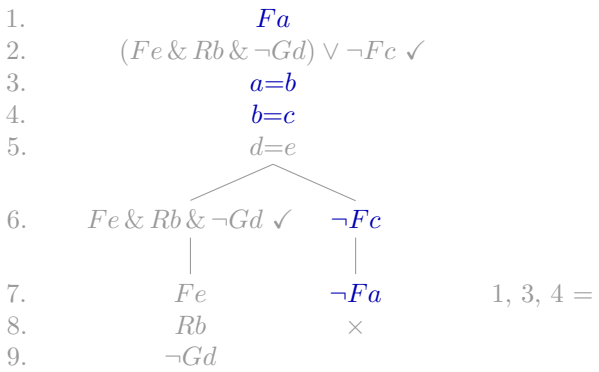
> A branch is COMPLETE if and only if either (i) it is closed, or (ii) all of the following conditions are met:
>
> 1. Every resolvable sentence in the branch has been resolved;
> 2. For every general quantified sentence Φ in the branch, at least one instance of Φ is in the branch; and, for every name $\mathfrak{a}$ in the branch, the $\mathfrak{a}$ instance of Φ is in the branch;
> 3. It is possible to select one name as the representative of each identity group in the branch such that, for every atomic sentence or negated atomic sentence in the branch other than a positive identity claim, its representative version is also in the branch.

In the example above, $a{=}b$ is in the branch, so $\{a, b\}$ are an identity group in the branch. We need to pick one of those two names as the representative, and ensure that we've done enough substitutions to ensure that the representative version of all atoms or negated atoms is present. If a is our representative, then we need every atom or negated atom containing b, other than identity claims, to have been transformed into an a sentence. But the branch contains $\neg Fb$ with no $\neg Fa$. So if a is the representative, the branch is incomplete. If b is the representative, we need every relevant line containing a to be transformed into a version with b; but the branch contains Fa but not Fb. Note that including either one of those developments would have closed the tree, just as we want.

The same principle applies if we have more than two names identified. For a more complicated example, consider this partially completed tree:

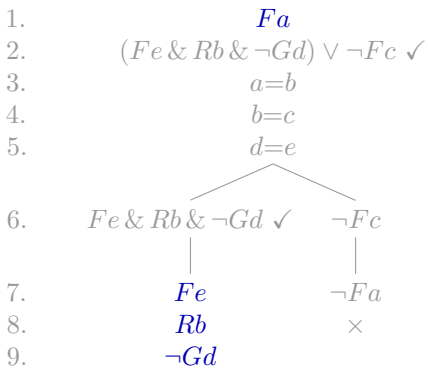


The three identity claims create two identity groups in both branches: $a{=}b$ and $b{=}c$ mean that a, b, and c are all identified with one another. And $d{=}e$ means that d and e are identified with one another. So to complete either branch, we need a representative of each of those identity groups to stand in for every other name it's identified with. Let's focus first on the right branch: there are two sentences that are atoms or negated atoms, other than the identity claims: Fa and $\neg Fc$. a and c are identified together, along with b. So we need to pick one of those three names as a representative, and substitute it for any other(s) in that branch. If we pick a, then we go from $\neg Fc$ to $\neg Fa$, and the branch closes:

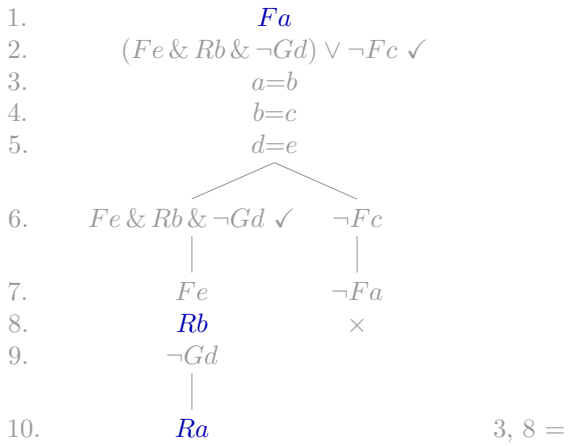


As a useful bit of practice, work out what would happen if one chose b or c as a representative instead. Either way, the branch will close. (If you pick b, you'll need one additional step.)

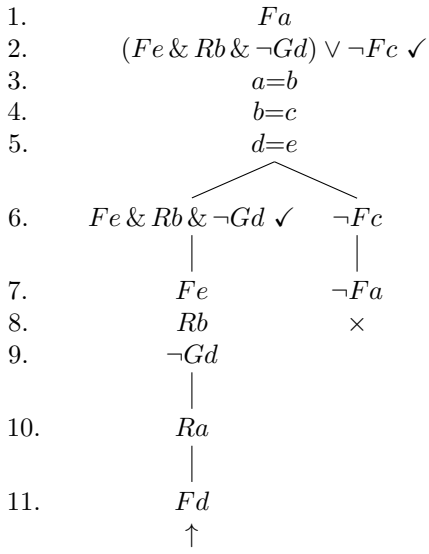
Now let's consider the left branch. Here, we'll need a representative from each of the two identity groups. Let's pick a as our representative for a, b, and c. So we need to check the branch for any atomic sentence or negated atomic sentence that contains a b or a c, other than an identity claim, and transform it into a representative version that uses a instead. The highlighted sentences are the ones we need to check:



The Rb at line 8 is the only place where we need to substitute a as a representative. (Our rules *allow* for this substitution for any sentence in the branch, but the completion rules *require* it only for atoms or negated atoms containing other identified names, which aren't identity claims.) Note that the $\neg Fc$ on line 6 is not in the open branch we're considering. So we'll develop the left branch with Ra:



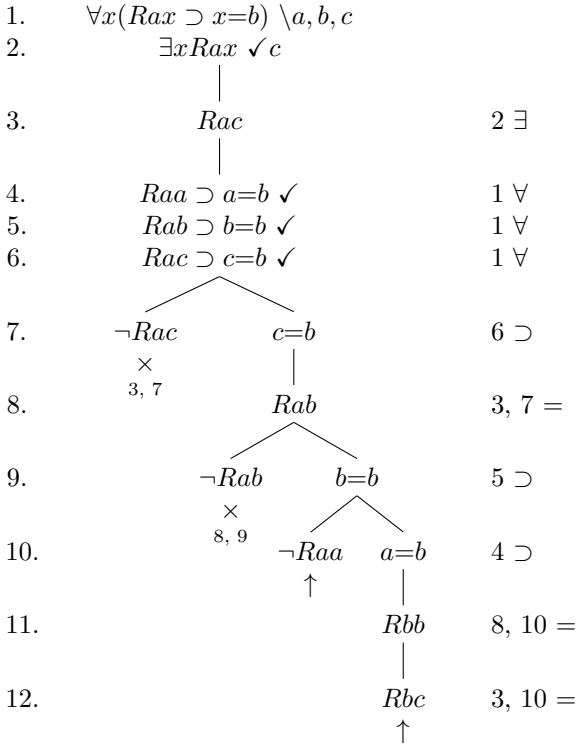
Finally, we need to make all necessary substitutions with a representative for the identity group with d and e. Let's pick d as the representative; we need to develop Fe on line 7 into Fd. There is no other non-identity atom or negated atom in the branch containing e, so once we've done that, our branch is complete:



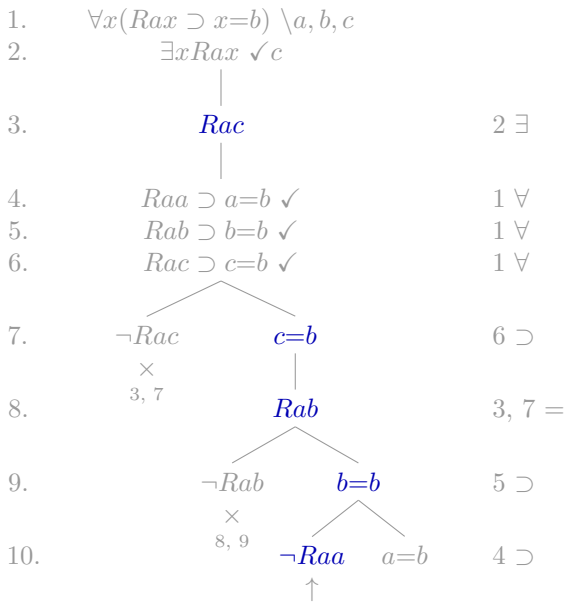
12.7.4 Constructing models with identity

Here is one last supplement to our tree method, having to do with constructing models from completed open branches. If a completed open branch includes any identity claims, the model must satisfy those claims by including only one object for each identity group in the UD, and having all names in the identity group refer to that object. For instance, if the sentence $b{=}c$ appears in an open branch, we will not construct a model with two different objects, b and c — instead, we'll just pick one of them as the object indicated, and assign both names to it.

Consider this tree:



We end up with two open branches. We only need one to demonstrate that the root is satisfiable, but it is useful practice to consider both. Let's begin with the shorter open branch to the left. Focus in on the atoms and negated atoms in that branch:



Examining that branch, we see three names, a, b, and c, and three atomic sentences concerning the extension of R: Rac, Rab, and $\neg Raa$. Using our previous method for constructing models, we would have posited a three-object UD and related those objects via R in the appropriate way. Now, however, we also have identity claims to consider. There are two identity claims in the branch. $b{=}b$ doesn't tell us anything interesting — of *course* that object is identical with itself — but the presence of $c{=}b$ indicates that our model will not include separate objects b and c. We'll only posit one object in the UD for those two names, and another object for a. We can pick either letter for the object; let's call it b. (Remember, it doesn't matter what kinds of objects we pick for the UD; it could be Reed Richards for all it matters. We typically pick letters because they are easy to keep track of.)

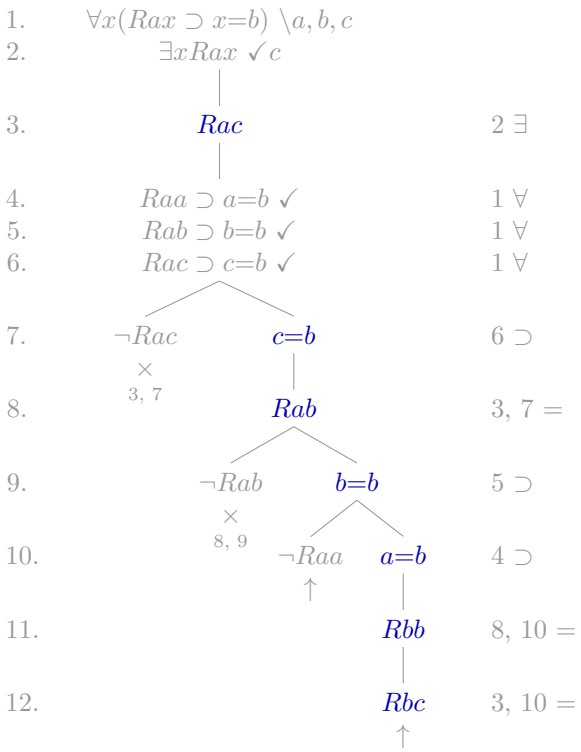
So the model suggested by the first open branch here is:

$$\begin{aligned} \text{UD} &= \{\text{a, b}\} \\ \text{referent}(a) &= \text{a} \\ \text{referent}(b) &= \text{b} \\ \text{referent}(c) &= \text{b} \end{aligned}$$

$\text{extension}(R) =$

Rxy	a	b
a	0	1
b	-	-

As a second example, let's consider the other, longer completed open branch in the same tree:



This branch includes both $a{=}b$ and $c{=}b$, which implies that all three of these names are names for the same object. So the UD for the model corresponding to the right branch will have only one object, which all three names denote.

$$\begin{aligned} \text{UD} &= \{\text{a}\} \\ \text{referent}(a) &= \text{a} \\ \text{referent}(b) &= \text{a} \\ \text{referent}(c) &= \text{a} \\ \text{extension}(R) &= \begin{array}{c|c} Rxy & a \\ \hline a & 1 \end{array} \end{aligned}$$

It is sometimes helpful, for an intuitive grip on what is going on in these trees, to think about an English translation alongside the formalisms. For example, suppose that Rxy is interpreted as 'x loves y'. Line 1 of the tree says that person a loves at most one person; line 2 says that a loves at least one person. The tree demonstrates two ways that might be: the one and only person a loves could be a, or it could be someone else.